

System Architecture

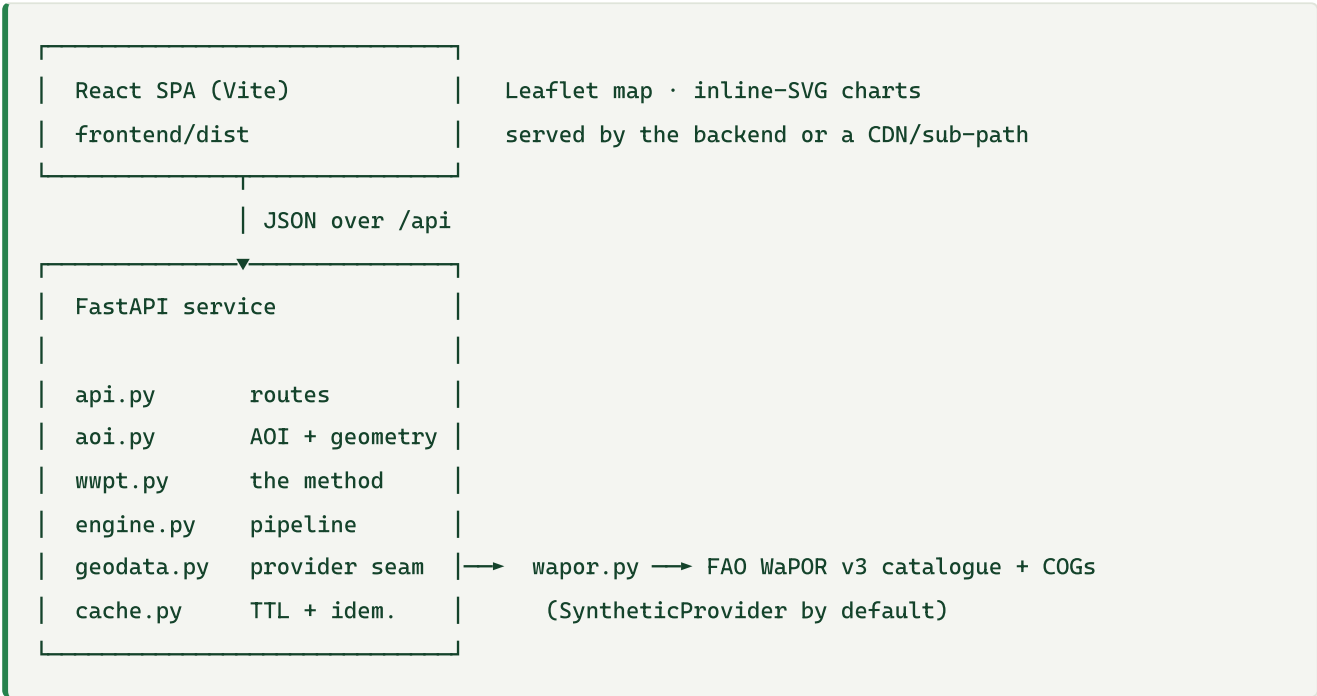
Wheat Water Productivity Dashboard

Ethiopian Institute of Agricultural Research · IWMI East Africa · FAO WaPOR Phase II
Generated 2026-08-19

Architecture

Overview

Two deployable pieces, no artifact store:



The frontend never talks to WaPOR directly; every value it renders comes from one of the nine `/api` routes. That boundary is what lets the data provider be swapped without touching the UI.

There is no model and no artifact store. The estimate is a closed-form calculation over two satellite variables, so what used to be a trained-model lifecycle is now a small, explicit parameter set (see **The method** below).

Request flow: one analysis run

1. **POST `/api/analysis`** with the AOI spec, production system, year and season. `Pydantic` validates types; `api._validate_season` rejects combinations the system cannot produce (for example Meher under irrigation).
2. **Idempotency and caching** (`cache.py`). If an `Idempotency-Key` header matches a completed run, that exact response is replayed — a retried POST never starts a second computation. Otherwise the

canonical request JSON is hashed and checked against a 30-minute TTL cache; a hit returns the stored result with `cached: true`.

3. **AOI resolution** (`aoi.resolve`). Admin units come from the boundary layer; drawn polygons are used directly; uploads are looked up by id in the upload registry. Anything unresolvable raises `AOIError`, surfaced as HTTP 422 with a message written for the person using the dashboard, not the developer.
4. **Grid and mask** (`engine.run_analysis`). A 170×170 lat/lon grid is built over the AOI bounds, intersected with the AOI geometry (vectorized even-odd point-in-polygon) and with the wheat-area mask.
5. **Season resolution** (`wwpt.season_window`). The (system, year, season) selection becomes concrete SOS and EOS dates — the window the reference notebook takes per plot. Reported with every result, so the assumption is visible rather than buried.
6. **Retrieval** (`geodata.PROVIDER.assemble`). Seasonal NPP and AETI for every unmasked cell as numpy arrays — one vectorized call, not per cell.
7. **Estimation** (`wwpt.estimate`). The biomass-to-yield chain applied to the whole array at once, returning every intermediate, not just the final value.
8. **Products**. The values are colourized and encoded as a PNG data-URI (`pnglib.py`, stdlib `zlib` only — no image dependency), plus zonal statistics, the class distribution, a five-season trend computed on a coarser 42×42 grid, and the estimation chain for the area mean. The run is registered under a `run_id` so CSV export can reproduce it.

Pixel inspection (`/predict`) and derivation (`/explain`) run the same retrieval and estimation for a single coordinate, so a pixel's popup value and its derivation are always consistent with the raster around it.

Request flow: one scheme run

The area-of-interest journey above answers "how productive is this extent?" over a continuous grid. The reference notebook answers a different question, the one an irrigation scheme actually asks: given a file of plots or sample points, each carrying its own growing season, what is the yield and water productivity of each one? `schemes.py` implements that second flow, and its input and output follow the notebook rather than this service's raster conventions.

1. **POST /api/upload** parses the file into features, keeping the DBF or GeoJSON attributes. Attributes are not cosmetic here: `SOS` and `EOS` drive the estimate, and `Name`, `Location` and `Scheme_ID` are what results are grouped by afterwards. The response reports the fields found and whether the file passes `wwpt.validate_features`, the port of `etwapor.util.validate_input`, so the interface can offer per-plot estimation only when it will actually work, and name the missing field when it will not.
2. **POST /api/schemes/analysis** with the upload id. Caching and idempotency behave as they do for the raster journey.
3. **Sampling** (`schemes._sample_points`). A point sample is evaluated where it sits. A boundary is evaluated on a 24×24 grid clipped to its ring, which is the zonal mean the notebook takes over the WaPOR cells inside the plot; a plot smaller than the grid step falls back to its centroid rather than returning nothing.
4. **Retrieval per feature** (`PROVIDER.assemble_window`). Each feature's own `[SOS, EOS]` window, so one file may mix seasons — the 2026 campaign file mixes four. Both providers implement this entry point,

so the workflow runs unchanged on synthetic or live WaPOR data.

5. **Estimation** (`wwpt.estimate`), then the five notebook columns appended to the attributes the feature arrived with: `NPP`, `EYield_tpha`, `AETI_mm`, `WP_kgpm3`, `LGP`. Water productivity is the mean of the per-cell ratios, not the ratio of the means: a ratio does not commute with averaging, and the mean of the per-cell values is the honest plot figure.
6. **Aggregation** (`schemes._median_rows`), for point input only. Samples are collapsed to one row per plot by median, grouped by `Name`, `Location`, `SOS`, `EOS`, `LGP`, `Scheme_ID` — notebook cell 26 exactly. The median rather than the mean, so one bad sample cannot move a plot.
7. **Products**. The per-feature table, the aggregated table, and the two figures the notebook plots (estimated yield and water productivity per scheme, drawn from the medians where they exist). `GET /api/schemes/export/csv` returns either table with those columns, matching `Irrigated_Wheat_WP_BND_2026.csv` and `Irrigated_Wheat_WP_PNT_2026.csv`.

Getting a file in. `GET /api/schemes/datasets` lists the scheme files this deployment can hand over, and `GET /api/schemes/dataset?name=...` returns one as GeoJSON. Where the machine holds the 2026 campaign shapefiles — `./Data`, or wherever `WWP_CAMPAIGN_DATA` points — the two real files are offered: `Irrigated_Wheat_2026` (6 scheme boundaries) and `Irrigated_Wheat_2026_PNT` (57 sample points), read from disk by `campaign.py` with the same reader that parses an upload. Elsewhere, and on any public deployment, only `schemes.sample_file`'s generated stand-in is listed, so the interface never offers a file it cannot produce. Nothing in the repository holds campaign geometry: keeping IWMI's field data out of it was deliberate, and embedding coordinates in code would undo that. The client posts whichever file it receives back through `POST /api/upload`, so a campaign file is parsed, validated and estimated by exactly the code an uploaded one is, and the ingest suite asserts the two routes agree.

One consequence is worth stating because it changed existing behaviour: the 3°×3° extent cap is a property of the raster journey, not of an uploaded file. Gridding costs $O(\text{area})$, so an unbounded AOI would attempt the whole country at 100 m; per-feature estimation costs $O(\text{features})$ and samples each plot in place. The cap therefore moved from `register_upload` to `aoi.resolve`. Without that move the reference notebook's own input would be rejected on upload, since its schemes span Afar to Oromia, about 3.5° of latitude.

The method

`wwpt.py` is a port of `etwapor.productivity.estimate_wheat_wp` from the IWMI reference notebook `ETH_WWP_WaPORv3_Irrigaed_Wheat_2026_Final.ipynb`:

```
TB = AOT · fc · NPP · 22.222 / (1 - mc)    total biomass, kg DM/ha
Y   = TB · hi                             grain yield, kg/ha
CWP = Y / SWC, SWC = AETI · 10             water productivity, kg/m³
```

NPP arrives in gC/m²/season and AETI in mm/season. 22.222 converts gC/m² to kg dry matter per hectare (1 gC/m² = 10 kgC/ha; dry matter is ~45% carbon, so 10/0.45); 1 mm over a hectare is 10 m³.

Crop parameters. AOT 0.85, fc 0.90, mc 0.15, hi 0.48, taken from `etwapor.data.wheat` once IWMI supplied the package. Their product, `AOT · fc · hi / (1 - mc) = 0.4320`, is the only combination the outputs depend on. Before the package was available these four were inferred from the notebook's

published records, which pin the product but not the individual values; the inferred set had the same product and therefore the same results, but would have misled anyone changing one parameter in isolation. That risk is gone. `WWP_CROP_PARAMS` (or `backend/crop_params.json`) overrides them; unknown keys are rejected rather than ignored, so a typo cannot silently leave a default in place.

Parity. Two suites, covering the two halves of "agrees with the reference".

`tests/test_notebook_parity.py` checks the arithmetic: it replays all 16 published records and requires the same yield and water productivity at the notebook's precision, plus the LGP day counts. It needs no server and no network, and it gates the Docker build — an image whose estimates disagree with the reference cannot be produced. `tests/test_campaign_files.py` checks the ingest: given the notebook's own 2026 campaign shapefiles it requires the same LGP per feature and the same six plots from the 57 sample points, grouped as notebook cell 26 groups them, and it requires the in-app loader described below to produce feature for feature what uploading the zipped shapefile produces. Those shapefiles are IWMI field data and are not committed, so that suite skips unless `WWP_CAMPAGN_DATA` points at them (it defaults to `./Data`).

Water productivity, plot versus area. `schemes.py` divides the plot's mean yield by its mean AETI, which is what `etwapor.productivity` does. `engine.py` averages the per-cell ratios instead, because over an extent the distribution of per-cell productivities is the thing being summarised. A ratio does not commute with averaging, so the two differ slightly; each module states which convention it follows and why.

Explanations. With a deterministic method there is nothing to attribute statistically: the explanation is the derivation. `engine.estimate_chain` returns each measured input, each parameter applied and each intermediate, and the UI renders them as a chain the reader can check by hand. Every step but the last is linear in NPP, so applying the chain to a mean NPP gives exactly the mean biomass and the mean yield; water productivity is a ratio and does not commute with averaging, so the area figure is the mean of the per-cell ratios and the panel says so.

Data-access layer

`geodata.PROVIDER` is the seam between the analytics and the outside world. It exposes one method the engine depends on:

```
assemble(lat, lon, system, year, season) -> {"npp", "aeti", "sos", "eos", ...}
```

Two implementations satisfy it, selected by `WWP_PROVIDER`.

SyntheticProvider (default) builds fields from sums of harmonics: values are deterministic (the same coordinates always give the same answer) and spatially coherent (neighbouring cells correlate, as real remote-sensing products do). It generates seasonal water consumption and a biomass water-use efficiency, then multiplies them, so NPP and AETI covary the way they do in the real products instead of independently — and the results land in the range the reference notebook observed over the 2026 Ethiopian schemes (NPP 65–560 gC/m², AETI 190–460 mm). Everything it produces is flagged `synthetic: true` through to the interface: a number the reader cannot distinguish from real WaPOR output is worse than no number.

WaporProvider (`wapor.py`) retrieves the real thing. FAO's GISManager v2 catalogue holds one raster per dekad, coded `WAPOR-3.{mapset}.{YYYY}-{MM}-D{n}`. Because the code is derived from the date, the

provider enumerates the dekads in the season window itself and asks for each raster by code rather than depending on query-filter syntax. Each entry carries a `downloadUrl` for a cloud-optimised GeoTIFF; those are ~620 MB each and are never downloaded — the store advertises `Accept-Ranges: bytes`, so GDAL reads only the tiles covering the requested points through `/vsicurl/`. Values are scaled integers, with the scale and offset taken from the *mapset* metadata rather than hard-coded. Dekadal products are daily rates, so each dekad contributes `rate × days`, and dekads clipped by the season boundary contribute only their overlapping days — which is what lets an arbitrary SOS/EOS pair resolve exactly.

One deliberate divergence from the reference retrieval. `etwapor.download._get_storage_url` builds every dekadal filename with a literal `D1` suffix (`f'...{date_str[0:8]}D1{ext}'`, where the slice keeps only `YYYY-MM-`), so all three dekads of a month resolve to the same raster — the month's first — while `_compute_days` still weights them 10, 10 and 8–11 days. The three rasters are distinct and all published; `2025-11-D1`, `-D2` and `-D3` each return 200 from the storage bucket with different lengths. `wapor.py` therefore builds the requested dekad, which is what a seasonal sum needs. The consequence: **running the WaPOR provider will not reproduce the NPP and AETI figures the notebook printed**, and the difference belongs to the reference. Everything downstream of those two numbers agrees exactly, which is what `tests/test_notebook_parity.py` demonstrates by feeding the notebook's own NPP and AETI through this service's arithmetic. This should go to IWMI before the campaign figures are published.

Resolution. The provider reads the **L2 national mapsets at 100 m** (`L2-NPP-D`, `L2-AETI-D`), which is what `etwapor` reads. An earlier version of this module served L1 at 300 m, because the catalogue's *mapset listing* returns 22 entries and every one of them is L1. That listing is incomplete: fetching `/mapsets/L2-NPP-D` by code returns it in full, captioned "National - Dekadal - 100m" with scale 0.001, and the rasters resolve on the storage bucket. Level is set by `WWP_WAPOR_LEVEL`; L3 scheme mosaics need `WWP_WAPOR_SCHEME` (`KOG`, `AWH`) and are addressed under `MOSAICSET`. The dashboard reports whatever `PROVIDER.resolution_m` says rather than hard-coded copy.

Verification. `GET /api/wapor/check` re-runs the catalogue contract — the configured dekadal rasters exist on storage, and `measureUnit` matches what the accumulation assumes wherever the catalogue lists the mapset — and reports whether `rasterio` is installed. The catalogue half is exercised by the e2e suite against the live service. The pixel read itself has *not* been executed, because `rasterio` (GDAL) is a heavy binary dependency deliberately left out of `requirements.txt`; run the check before trusting any figure from this provider.

Visualization palette

Every chart colour was chosen by job and validated, not by eye. The validator checks lightness ordering, chroma, colour-vision-deficiency separation and contrast against the surface.

JOB	CHART	TOKENS
Sequential / ordinal (magnitude)	map raster, legend, distribution	#93bd82 #6ba763 #458b4b #297038 #0f4d26
Single series (identity)	trend line	#297038

The WWP ramp is single-hue green, not brown→yellow→green. The original HTML prototype's ramp put a pale yellow (`#f7dd72`) at the middle of the scale, which measures 1.32:1 against a white card — effectively invisible in the distribution chart — and its lightness was non-monotone, so the ramp did not

read as ordered under colour-vision deficiency or in greyscale print. A single green hue, light to dark, keeps the EIAR identity, encodes magnitude correctly, and clears the 2:1 light-end contrast floor. The backend `engine.RAMP` and the frontend `charts.RAMP_HEX` hold the same five values, so the map, the legend and the chart cannot drift apart.

The ramp spans a fixed 0.4–1.6 kg/m³ rather than stretching to each result's own range, so two *woredas* can be compared directly across runs. The cost is lower contrast within a single scene; for a monitoring tool comparability is worth more.

A diverging pair (orange/blue) previously encoded "raises"/"lowers" in the model explanation. It went with the model: the estimation chain has no polarity to encode, only an ordered sequence of quantities, and is rendered as text rather than marks because every element in it is a named number — there is no magnitude comparison a mark would make clearer.

Both remaining charts ship a **Show table** toggle rendering the same numbers as text, and a hover tooltip. No value is reachable only by hovering, which keeps the figures usable by keyboard and screen-reader users and satisfies the ToR's accessibility requirement.

Frontend

React with Leaflet driven imperatively. `MapView.jsx` owns every Leaflet call; React state flows in as props and Leaflet layers are reconciled in effects. Two details worth keeping:

- **The drawing preview updates in place.** Recreating the polyline and vertex markers on each new vertex changes the DOM node under the pointer, and Chromium will not emit `dblclick` when the target changes between the two clicks — so "double-click to finish" silently did nothing. The preview now mutates one persistent polyline via `setLatLngs` and appends only new markers.
- **Finishing a polygon has a button, not only a double-click.** Double-click is unavailable to keyboard users and awkward on touch, so `Finish polygon` is the primary control and the double-click is a shortcut.
- **The map watches its own container for resizes.** Leaflet only listens for *window* resizes. The results panel mounting or unmounting resizes `.mapwrap` through the flex layout without any window event, which would leave the map rendering at its stale width — blank tile gutters and a misplaced raster overlay — until the next pan. A `ResizeObserver` on the map element calls `invalidateSize()` instead, which also covers the responsive breakpoint.

The results panel is **mounted only once a run has produced results**, so it never occupies the layout while empty and the map gets the full width until there is something to show. Closing it returns that width and leaves a `Show results` control on the map, so dismissing the panel cannot strand a completed run.

Charts are hand-written inline SVG rather than a charting library: two small fixed-purpose figures cost less to draw directly than the ~150 kB a general charting bundle would add, and it keeps full control of the accessibility attributes and the mark specs.

Deployment notes

Single process serves both tiers: `uvicorn app.main:app` mounts `frontend/dist` at `/` when it exists, with the API at `/api`. `vite.config.js` sets `base: './'`, so the built bundle works unchanged from any sub-

path of the EIAR site (`/research-tools/www/` , say) behind a reverse proxy.

The service prints its active data provider at startup, loudly when it is the synthetic one, so a deployment cannot serve demonstration figures without that being visible in the logs as well as in the interface.

For production hardening beyond this build, see [the ToR Coverage document](#) § What deployment still needs — chiefly PostGIS for the upload registry and run store (both in-memory today), Redis for the cache if more than one worker runs, and enabling the WaPOR provider.